

Robust Mini Linear Servo Actuator —

mightyZAP 아두이노 API 매뉴얼



Size Comparison

mightyZAP

Powerful Mini Linear Servo Motors



목 차

<u>01 아두이노 라이브러리 활용</u>	<u>3</u>
1.1. 라이브러리 추가-----	3
1.2. 예제 불러오기-----	4
1.3. 프로그램 업로드 하기-----	4
<u>참고 아두이노 개발환경 구축하기</u>	<u>5</u>
아두이노 IDE 설치-----	5
아두이노 IDE 기본 구성-----	8
서보 회로 연결-----	9
<u>02 아두이노 MightyZAP API</u>	<u>11</u>
2.1. MightyZAPAPI 개요-----	11
2.2. MightyZAP API List -----	11
<u>03 아두이노 IDE 를 통한 서보제어 예제</u>	<u>15</u>
3.1. 개요 -----	15
3.2. 예제 - Information Read (TTL/RS-485)-----	15
3.3. 예제- LED -----	16
3.4. 예제 - Servo ID -----	17
3.5. 예제 - Goal Position-----	18
3.6. 예제 - Present Position-----	19
3.7. 예제 - GoalSpeed-----	20
3.8. 예제 - GoalCurrent-----	21
3.9. 예제 - StrokeLimit -----	22
<u>04 아두이노 IDE 를 통한 서보 PWM 제어 예제</u>	<u>23</u>
4.1. 개요 / 주의사항-----	23
4.2. 예제 - PWM 을 이용한 위치 제어 -----	23
<u>[참고 - 아 두 이 노 테스트 쉼드]</u>	<u>24</u>

1

아두이노 라이브러리 활용

아두이노 API는 당사 웹사이트(<http://www.irrobot.com/>) 자료실에서 다운로드가 가능합니다. MightyZAP 아두이노 API를 다운로드하여 아두이노 IDE에서 서버 액추에이터 제어가 가능합니다.

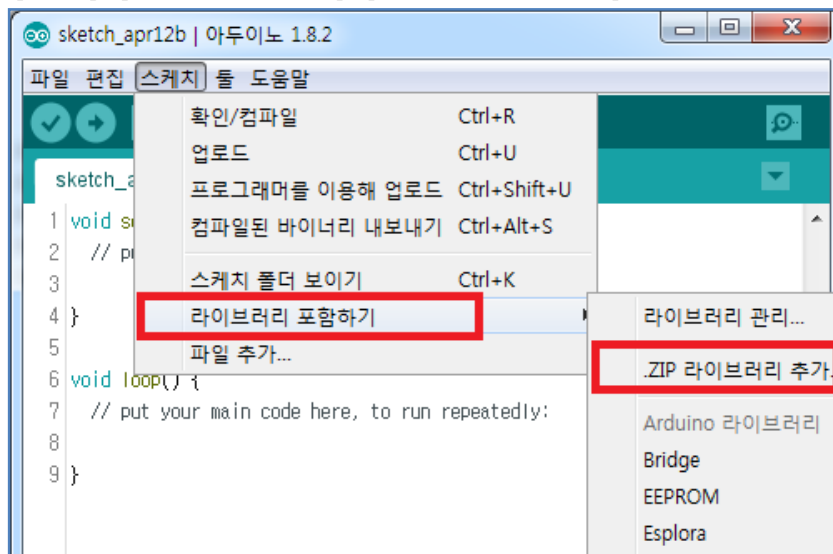
당사에서 제공한 아두이노 API는 아두이노 레오나르도 기준입니다. 기타 다른 아두이노 사용자분들께서는 이를 감안하여 적절히 API를 수정하여 사용하여 주십시오.

MightyZAP API를 사용하기 위해서는 다음의 과정을 수행해야 합니다.

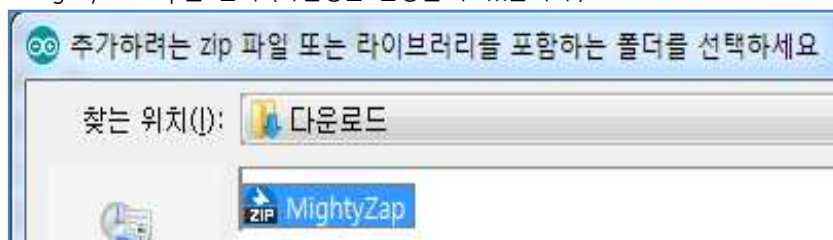
1. 아두이노 IDE MightyZAP 관련 라이브러리 추가
2. MightyZAP 관련 예제 불러오기
3. 프로그램 업로드 하기

1.1 라이브러리 추가

1. 웹사이트 자료실에서 MightyZAP.Zip을 내려 받습니다.
2. [스케치] - [라이브러리 포함하기] - [.ZIP 라이브러리 추가...] 선택



3. MightyZAP.Zip을 선택 (파일명은 변경될 수 있습니다.)



1.2 예제 불러오기

1. 아두이노 IDE 실행
2. [파일] - [예제] - [mightyZAP] - 사용할 예제 선택

1.3 프로그램 업로드 하기

Arduino 보드의 종류에 맞는 예제를 선택하여 업로드 합니다.

UNO 계열 보드 일 때

[파일] - [예제] - [mightyZAP] - [UNO]- [MightyZap_Information]

Leonardo 계열 보드 일 때

[파일] - [예제] - [mightyZAP] - [LEO]- [MightyZap_Information]

[참고 - 아두이노 개발 환경 구축하기]

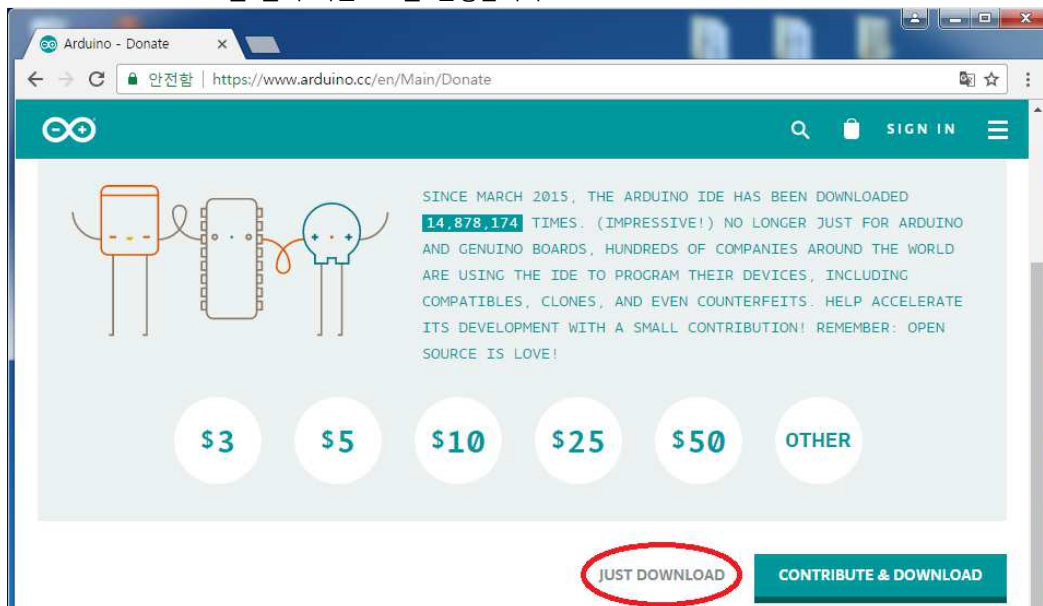
IRROBOT Servo Tester Shield는 Arduino Uno 또는 Arduino Leonardo에 결합하여 사용합니다. 테스트 프로그램을 동작시키기 위해선 PC에 아두이노 개발 환경을 구성해야 합니다.

1 아두이노 IDE 설치

1. <https://www.arduino.cc/en/main/software> 에서 Window Installer를 선택



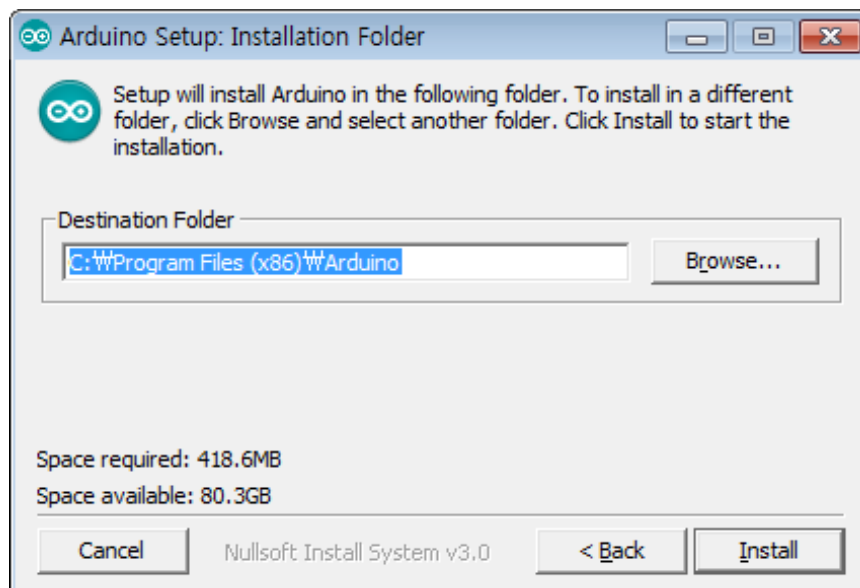
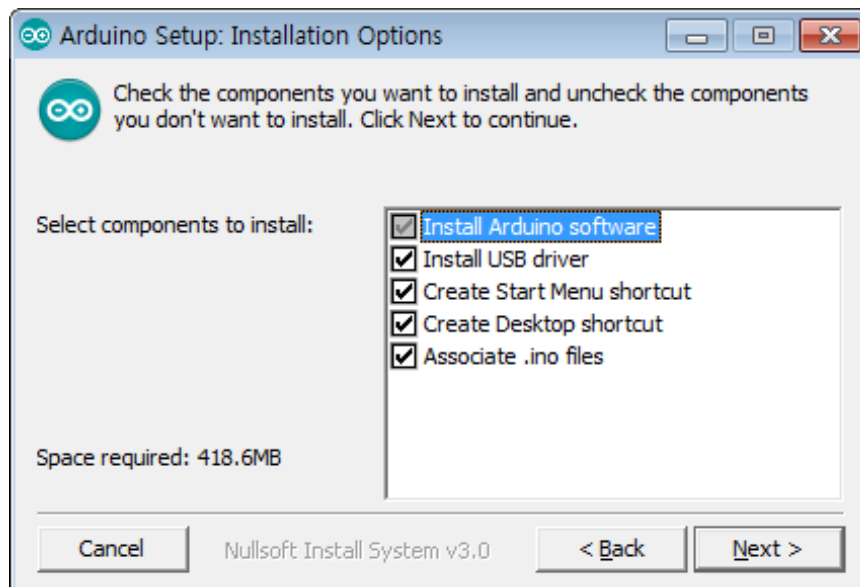
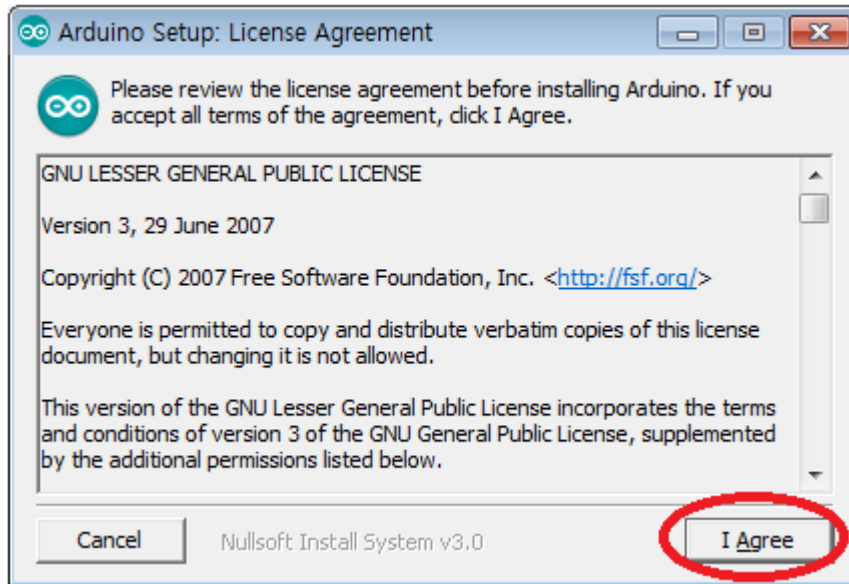
2. JUST DOWNLOAD를 눌러 다운로드를 진행합니다.

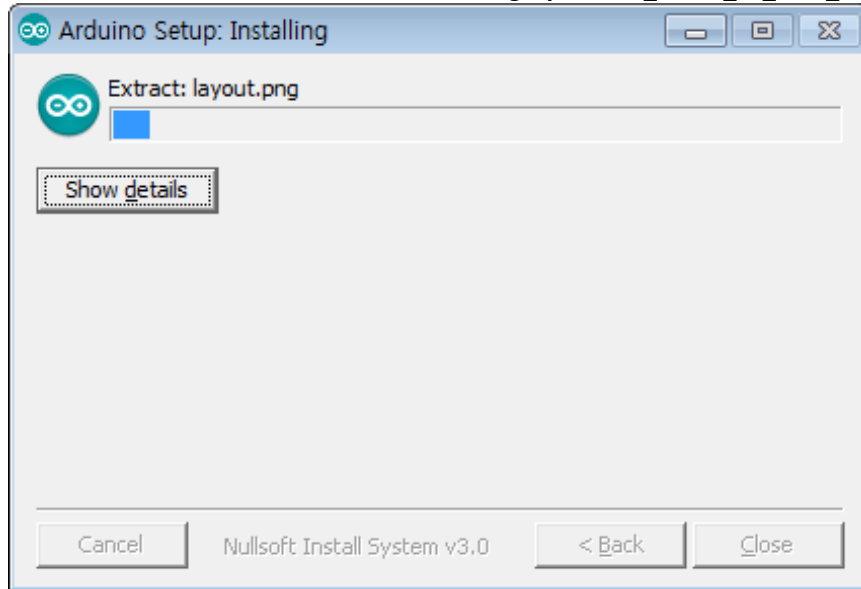


3. 다운로드가 완료 되면 arduino-1.8.2.exe를 실행 합니다.

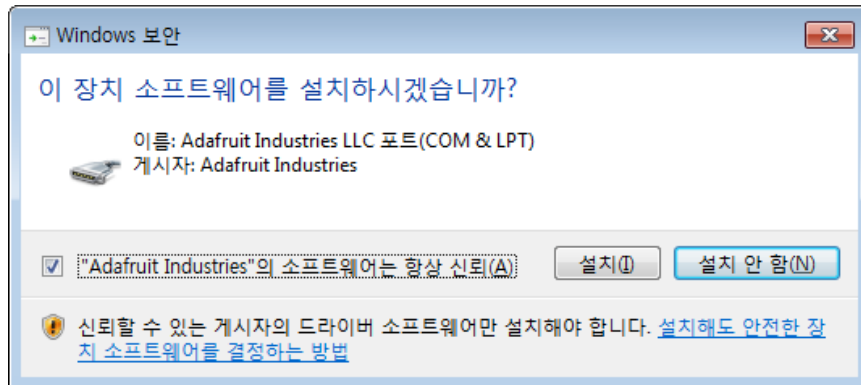


4. 다음의 과정을 따라 소프트웨어를 설치를 진행 합니다.

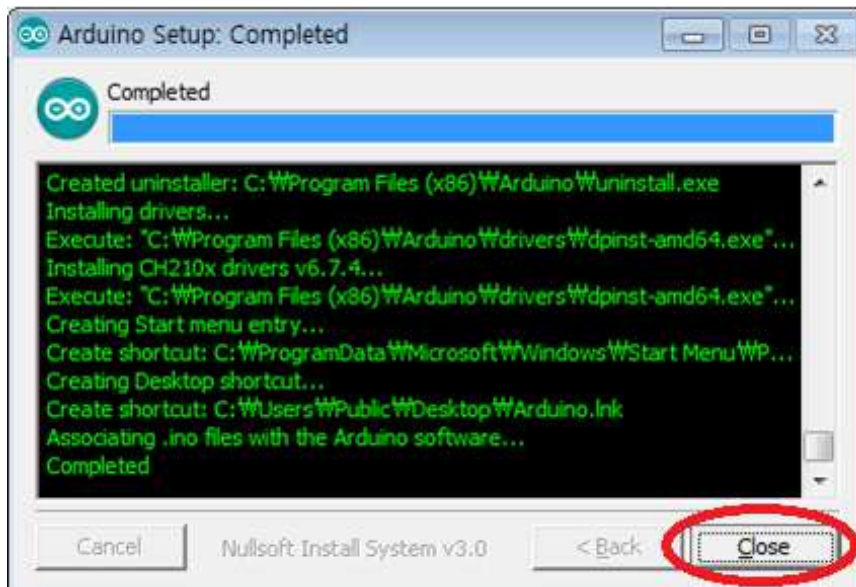




5. 설치 과정에서 나오는 추가 드라이버를 설치합니다.



6. 소프트웨어 설치가 완료되면 Close 를 누릅니다.

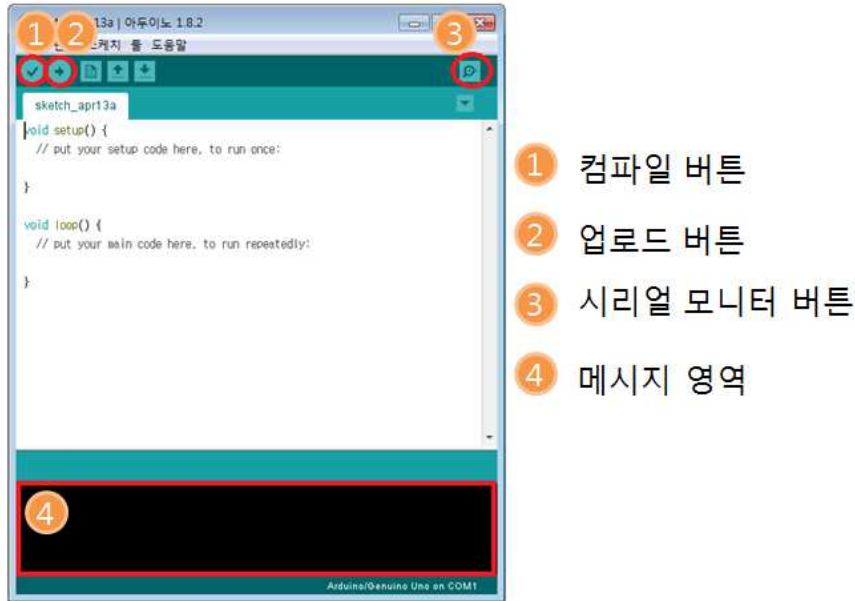


7. 바탕화면에 생성된 아두이노를 실행 합니다.



2.아두이노 IDE 기본 구성

아두이노 개발환경 기본 구성은 다음과 같습니다.



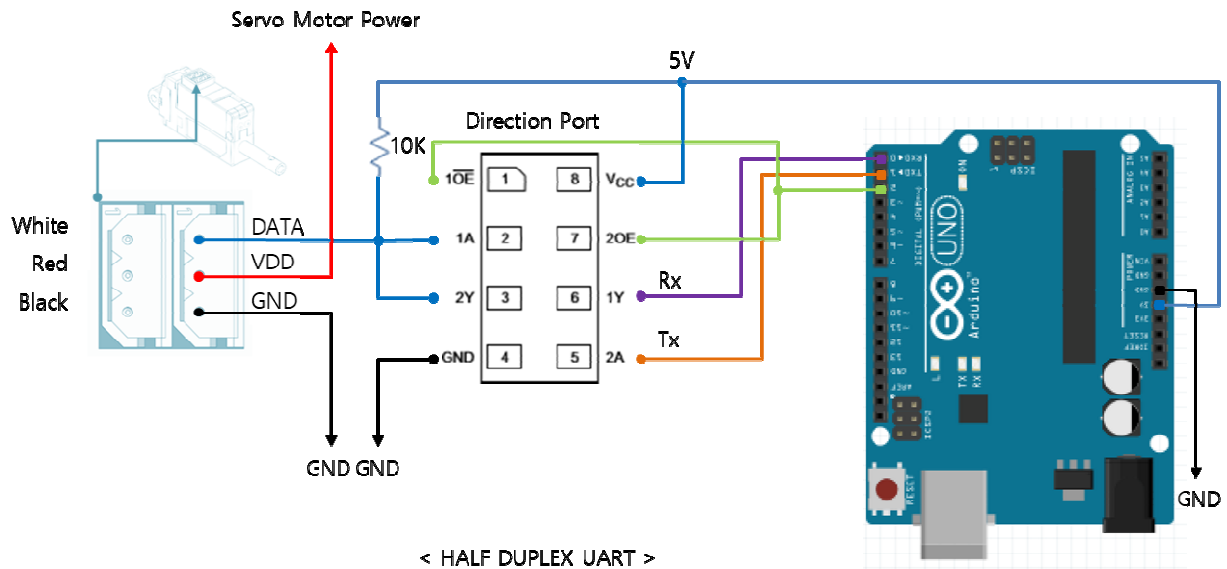
1. 컴파일 버튼 : 이 버튼을 누르면 프로그램이 컴파일 합니다.(컴파일 결과는 4번에 표시)
2. 업로드 버튼 : 컴파일과 동시에 Arduino에 업로드 합니다. (컴파일 오류 또는 Arduino와의 USB 연결 오류가 발생하면 4번 영역에 표시)
3. 시리얼 모니터 버튼 : USB로 아두이노와 연결된 상태로 아두이노가 동작하면 아두이노에서 PC로 메시지를 보낼 수 있습니다. Serial.write() 또는 Serial.print()같은 함수를 사용하여 프로그램이 작성된 경우 시리얼 모니터를 통해 메시지를 확인할 수 있습니다.
4. 메시지 영역 :아두이노 개발환경을 사용하면서 발생하는 알림 메시지나 컴파일, 업로드 결과 등이 표시되는 영역입니다.

3. 서보 회로 연결

MightyZAP서보액추에이터와 통신을 하려면UART 신호를 Half Duplex type로 변환 시켜줘야 합니다.이때 변환 회로는 아래 그림과 같습니다.(당사의 MightyZAP Tester Sheild IR-STS01 아래의 회로를 포함하고 있으므로 추가로 구성할 필요가 없습니다.) 데이터 통신이 아닌 PWM 제어를 원하는 사용자는 4장을 참고하십시오.

3.1. TTL (3핀 커넥터)

MightyZAP(PT)을 TTL레벨로 제어 하기 위해서 제어기의 UART 신호를 Half Duplex type로 변환 시켜야 합니다.이때 변환 회로는 아래 그림과 같습니다.



위의회로도에서 TTL Level의Tx와 Rx는 DIRECTION_PORT의 Level에따라다음과같이 Data 신호의방향이결정됩니다.

- DIRECTION_PORT의신호 Level이LOW인경우 :Data의신호가 Rx로입력
- DIRECTION_PORT의신호 Level이HIGH인경우 :Tx의신호가 Data로출력

※ 위 회로대로 구성을 했을 경우 서보 액추에이터와의 통신을 연결하기 위한 명령어로MightyZAP m_zap(&serial, pin, Tx_Lvel)을 사용해야합니다. 이때 &Serial을 아두이노의 해당 Serial, pin의 경우 Direction Port의 pin 번호를, Tx_Level의 경우 1로 설정해 주시면 됩니다. (위의 그림의 경우 MightyZAP m_zap(&serial, 2, 1);)

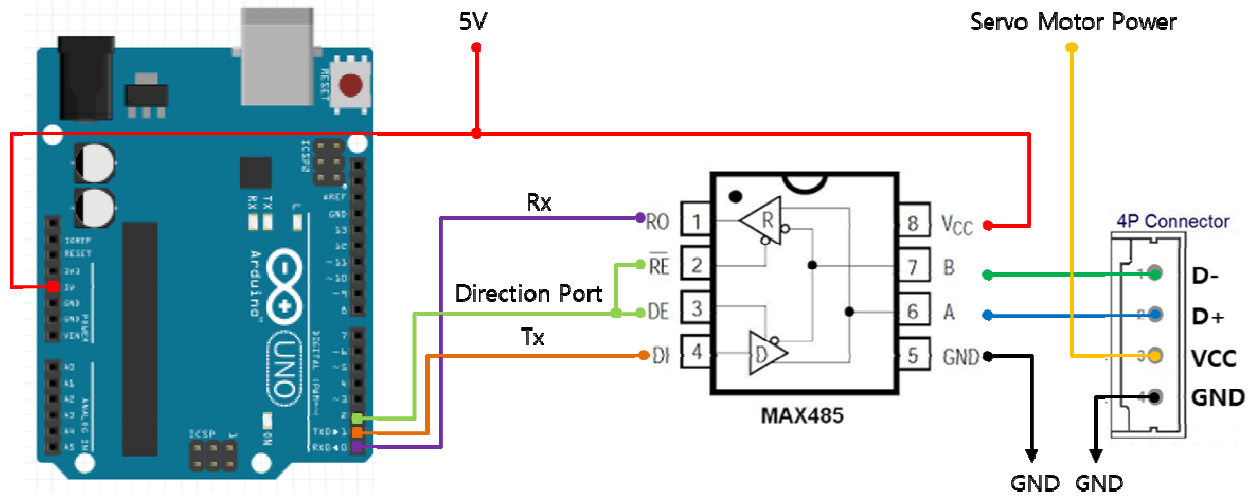
3.2 RS-485 (4 핀 커넥터)

mightyZAP (F)을제어하기위해RS485 통신방식을 사용 합니다.

이때 핀맵과 변환회로는 다음 페이지의 그림과같습니다.

PIN NUMBER(COLOR)	PIN NAME	FUNCTION(RS485)
1(황색)	D-	RS485 -
2(백색)	D+	RS485 +
3(적색)	VCC	전원 +
4(흑색)	GND	전원 -

ARDUINO mightyZAP API_for PC_FC_V1.0_KR



※전원이 별도로 외부에서 공급되는 경우에는, 485 D+, 485 D-만 연결해도 동작에 문제가 없습니다.

위의 회로도에서DIRECTION_PORT핀을 제어하여 RS485의 TX및 RX 모드를 변환할 수 있습니다.

- DIRECTION_PORT의신호 Level이LOW인경우 :Data의신호가 Rx로입력
- DIRECTION_PORT의신호 Level이HIGH인경우 :Tx의신호가 Data로출력

※위 회로대로 구성을 했을 경우 서보 액츄에이터와의 통신을 연결하기 위한 명령어로MightyZAP m_zap(&serial, pin, Tx_Level)을 사용해야합니다. 이때 &Serial을 아두이노의 해당 Serial, pin의 경우 Direction Port의 pin 번호를, Tx_Level의 경우 1로 설정해 주시면 됩니다. (위의 그림의 경우 MightyZAP m_zap(&serial, 2, 1);)

2

아두이노 mightyZAP API

아두이노 IDE를 통해 데이터 통신 프로토콜(RS-485 또는 TTL)을 통해 서보 액추에이터를 제어합니다. 서보 액추에이터를 데이터 통신으로 제어하고 싶을 때 사용합니다.

2.1 API 개요

아두이노 보드를 통해 mightyZAP 을 제어하기 위한 통신용아두이노 API 입니다.

각 파라미터 기능에 대한 구체적인 설명은 마이티잡 사용자 매뉴얼을 참고하여 주십시오.

Memory 는 비휘발성으로 전원 off 시에도 저장된 값이 유지되지만 Parameter 값은 휘발성으로 전원 off 시 이전 Data 가 저장되지 않습니다. Parameter 값은 전원 인가시 default 값 또는 현재 상태 값을 나타냅니다.

2.2 API List

- Force Control 버전 액추에이터(모델명이 12Lf로 시작) 사용자들은 하기 모든 파라미터를 사용할 수 있습니다.
- **[F/C version only]**라고 표시된 파라미터는 Force Control 버전 액추에이터(모델명이 12Lf로 시작)에만 적용되는 파라미터입니다. Position control 버전 액추에이터에는 적용이 되지 않습니다.
- Position Control 버전 액추에이터(모델명이 L12 또는 D12로 시작) 사용자는 **[F/C version only]**표시된 파라미터를 제외한 모든 파라미터를 사용할 수 있습니다.

MightyZAP m_zap(&serial, pin) / IR-STS01 MightyZAP Sheild 사용시 통신설정

- 기능 : 서보 액추에이터와의 통신 포트 설정
- 파라미터
 - Serial : 아두이노 레오나르도 보드를 이용할 경우, Serial1 포트를 통해 서보 액추에이터와 통신합니다.
 - pin : GPIO 2번을 이용하여 송수신 을 전환합니다. (default : 2)

MightyZAP m_zap(&serial, pin, Tx_Level) /아두이노 보드만 사용 시 통신설정

- 기능 : 서보 액추에이터와의 통신 포트 설정
- 파라미터
 - Serial : 아두이노 레오나르도 보드를 이용할 경우, Serial1 포트를 통해 서보 액추에이터와 통신합니다.
 - pin : 해당 GPIO핀을 이용하여 송수신 을 전환합니다.
 - Tx_Level : RS485의 제어레벨 설정(High or Low)

m_zap.begin(int baudrate) /통신속도 설정

- 기능 : 서보 액추에이터의 통신속도 설정
- 파라미터
 - baudrate : 서보 액추에이터와의 통신 속도 설정

Data	설 명
16	115200 baudrate
32	57600 baudrate
64	19200 baudrate
128	9600 baudrate

m_zap.getModelNumber(int ID_NUM) / Parameter Read

- 기능 : 서보 액츄에이터의 모델 번호 확인
- 파라미터 - ID_NUM : 해당 서보 액츄에이터의 ID

m_zap.Version(int ID_NUM) / Parameter Read

- 기능 : 서보 액츄에이터의 버전 확인
- 파라미터 - ID_NUM : 해당 서보 액츄에이터의 ID

m_zap.PresentTemperature(int ID_NUM) / Parameter Read

- 기능 : 서보 액츄에이터의 현재 온도 확인 (온도는 정확하지 않으니 참고용으로만 사용하세요)
- 파라미터 - ID_NUM : 해당 서보 액츄에이터의 ID

m_zap.LED(int ID_NUM, int Color) / Parameter Write (Volatile)

- 기능 : 서보 액츄에이터의 LED 설정
- 파라미터
 - ID_NUM : 해당 서보 액츄에이터의 ID
 - Color : 서보 액츄에이터의 LED Color

Data	설 명
RED	RED ON
GREEN	GREEN ON
BLUE	BLUE ON [For Position Control lineup Only]

m_zap.ServoID(int ID_NUM, int ID_SET) / Memory Write (Non-Volatile)

- 기능 : 서보 액츄에이터의 ID 설정
- 파라미터
 - ID_NUM : 해당 서보 액츄에이터의 ID
 - ID_SET : 서보 액츄에이터의 수정 ID

m_zap.GoalPosition(int ID_NUM, int position) / Parameter Write (Volatile)

- 기능 : 서보 액츄에이터의 위치이동 명령 설정
- 파라미터
 - ID_NUM : 해당 서보 액츄에이터의 ID
 - Position : 0~4095까지의 목표 위치값 제어 (default : 0~4095)

m_zap.GoalSpeed(int ID_NUM, int speed) / MemoryWrite(Non-Volatile) [FC Version Only]

- 기능 : 서보 액츄에이터의 이동속도 값 설정
- 파라미터
 - ID_NUM : 해당 서보 액츄에이터의 ID
 - Speed: 0, 1 ~ 1023 값으로 서보 액츄에이터의 이동 속도를 설정합니다.
속도값은 1~1023으로 설정하며
0은 최고속도 값인 1023과 같다.

m_zap.GoalCurrent(int ID_NUM, int current) / MemoryWrite (Non-Volatile) [FC Version Only]

- 기능 : 서보 액츄에이터의 최대 전류값 설정
- 파라미터
 - ID_NUM : 해당 서보 액츄에이터의 ID
 - current : 1 ~ 1600 값으로 서보 액츄에이터의 최대 전류값을 설정합니다.
설정값의 단위는 mA이며 오차를 포함하고 있습니다.
설정 값이 작으면 모터의 Force와 Speed 줄어듭니다.

m_zap.presentPosition(int ID_NUM) / Parameter Read

- 기능 : 서보 액츄에이터의 현재 위치 읽어오기
- 파라미터
 - ID_NUM : 해당 서보 액츄에이터의 ID

m_zap.moving(int ID_NUM) / Parameter Read

- 기능 : 서보 액츄에이터의 동작상태 읽어오기
- 파라미터
 - ID_NUM : 해당 서보 액츄에이터의 ID

m_zap.ShortStrokeLimit(int ID_NUM, int position) / MemeoryWrite(Non-Volatile)

- 기능 : 서보 액츄에이터의 최소 위치한계값 설정
- 파라미터
 - ID_NUM : 해당 서보 액츄에이터의 ID
 - position: 서보 액츄에이터의 최소 위치 한계 값 설정 (0~4095)

m_zap.LongStrokeLimit(int ID_NUM, int position) / MemeoryWrite (Non-Volatile)

- 기능 : 서보 액츄에이터의 최대 위치한계값 설정
- 파라미터
 - ID_NUM : 해당 서보 액츄에이터의 ID
 - position: 서보 액츄에이터의 최대 위치 한계 값 설정 (0~4095)

m_zap.Acceleration(int ID_NUM, int acceleration) / Memeory Write (Non-Volatile) [FC Version Only]

- 기능 : 액츄에이터의 가속률 설정
 - 파라미터
 - ID_NUM : 해당 서보 액츄에이터의 ID
 - acceleration : 서보 액츄에이터의 가속률값 설정 (0~255)
- ※값이 클수록 가속도가 빨라지고, 값이 작아질수록 부드러운 Start를 보입니다.

m_zap.Deceleration(int ID_NUM, int deceleration) / Memeory Write (Non-Volatile) [FC Version Only]

- 기능 : 액츄에이터의 감속률 설정
 - 파라미터
 - ID_NUM : 해당 서보 액츄에이터의 ID
 - deceleration : 서보 액츄에이터의 감속률값 설정 (0~255)
- ※값이 작아질수록 부드러운 정지를 보여줍니다.
 ※너무 큰 값으로 설정하면 최종 위치 도달 시 Overshoot 및 진동이 발생할 수 있습니다.
 ※Speed 및 GoalCurrent 등 액츄에이터의 기본 설정을 변경 중 Overshoot이 발생하면 감속률을 줄여주면 됩니다.

m_zap.forceEnable(int ID_NUM, int value) / Parameter Write (Volatile)

- 기능 : 서보 액츄에이터의 force On /Off
 - 파라미터
 - ID_NUM : 해당 서보 액츄에이터의 ID
 - value : servo Motor의 force를 On/ Off 합니다.
- Force off시 모터는 동작하지 않으며 GoalPosition 명령 또는 Force On 명령시 동작됩니다.
- 0 : Force Off
 1 : Force On

당사 서보 액츄에이터 중 35N이상의 제품의 경우 모터의 전원이 해제되어도 기구적인 설계특성상 위치를 고수하려는 특성이 있습니다.
 따라서, 설비에서 서보 액츄에이터가 특정위치를 지속적으로 고수하고 있어야하는경우 Force Off 명령으로 모터전원을 차단하여 모터의 수명을 연장시킬 수 있습니다. 이 경우 통신은 여전히 유지되며, 모터의 전원은 차단됩니다. 다시 위치이동명령을 내리게 되면 자동으로 Force ON 되어 다음 명령을 수행하게 됩니다.

m_zap.readbyte(int ID_NUM, int Addr)

- 기능 : Data Map의 Memory/Parameter의 1byte Data를 읽기 위해 해당 주소로부터 직접 읽어오기
- 파라미터
 - ID_NUM : 해당 서보모터의 ID
 - Addr : Data를 읽어올 파라미터 주소

m_zap.readint(int ID_NUM, int Addr)

- 기능 : Data Map의 Memory/Parameter의 2byte Data를 읽기 위해 해당 주소로부터 직접 읽어오기
- 파라미터
 - ID_NUM : 해당 서보모터의 ID
 - Addr : Data를 읽어올 파라미터 주소

m_zap.writebyte(int ID_NUM, int Addr, int Data)

- 기능 : Data Map의 Memory/Parameter의 1byte Data를 쓰기 위해 해당 주소에 직접 입력
- 파라미터
 - ID_NUM : 해당 서보모터의 ID
 - Addr : Data를 입력할 파라미터 주소
 - Data : 입력 Data(0~255)

m_zap.writeint(int ID_NUM, int Addr, int Data)

- 기능 : Data Map의 Memory/Parameter의 2byte Data를 쓰기 위해 해당 주소에 직접 입력
- 파라미터
 - ID_NUM : 해당 서보모터의 ID
 - Addr : Data를 입력할 파라미터 주소
 - Data : 입력 Data(0~4096)

3

아두이노 IDE 를 통한 서보제어 예제

3.1 개요

아두이노 보드를 통해 mightyZAP 을 제어하기 위한 아두이노 예제입니다. 본 예제는 시리얼 데이터 통신(TTL 혹은 RS-485 통신)을 통한 제어가 가능하며 PWM 통신은 지원하지 않습니다. (PWM 제어는 4 장 참조)

당사에서 제공한 아두이노 API는 아두이노 레오나르도 기준입니다. 기타 다른 아두이노사용자분들께서는 이를 감안하여 적절히 API를 수정하여 사용하여 주십시오. 각 파라미터 기능에 대한 구체적인 설명은 마이티잡 사용자 매뉴얼을 참고하여 주십시오.

당사의 아두이노 테스트 쉴드(IR-STS01)를 사용하면 보다 편리하고 효율적으로 아두이노 제어가 가능합니다.

3.2. 예제 - Information Read (RS-485/TTL)

MightyZAP의 API를 이용하여 ID, Version등 기본적인 정보들을 읽어오는 예제입니다.

[예제] - [MightyZAP] - [UNO/LEO]-[MightyZAP_Information] 선택

```

MightyZap_Information $
#include <MightyZap.h>

#define ID_NUM 0

MightyZap m_zap(&Serial1, 2);

void setup() {
  Serial1.begin(9600);
  m_zap.begin(32);
  while (! Serial1);
}

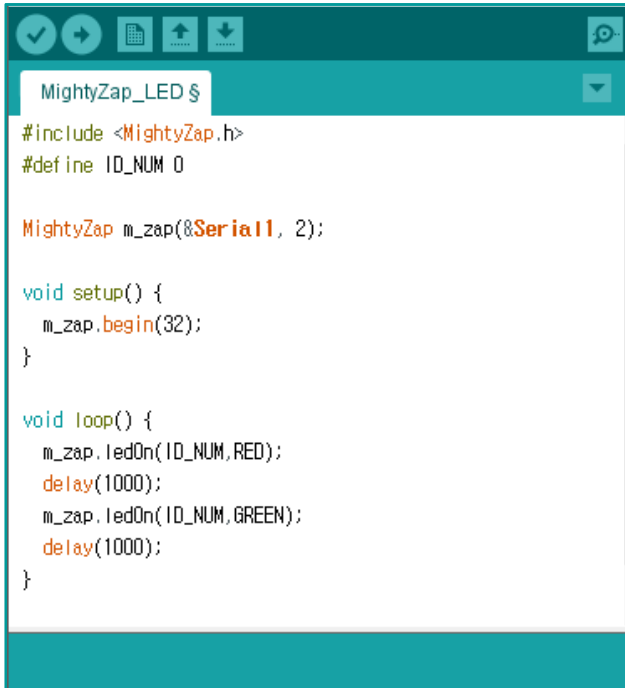
void loop() {
  if(Serial1.available()) {
    char ch = Serial1.read();
    if(ch=='d') {
      Serial1.print("Model Number      : "); Serial1.println((unsigned int)m_zap.getModelNumber(ID_NUM));
      Serial1.print("Firmware Version    : "); Serial1.println(m_zap.Version(ID_NUM)+0.1);
      Serial1.print("Present Temperature : "); Serial1.println(m_zap.presentTemperature(ID_NUM));
    }
  }
}

```

3.3. 예제 - LED

MightyZAP의 LedON() 명령어를 이용하여 서보 액추에이터의 LED를 설정합니다.

[예제] - [MightyZAP] - [UNO/LEO]- [MightyZAP_LED] 선택



```

MightyZap_LED $
#include <MightyZap.h>
#define ID_NUM 0

MightyZap m_zap(&Serial1, 2);

void setup() {
  m_zap.begin(32);
}

void loop() {
  m_zap.ledOn(ID_NUM, RED);
  delay(1000);
  m_zap.ledOn(ID_NUM, GREEN);
  delay(1000);
}

```

Description

LedON() 명령어를 이용하여 LED를 동작 시킬 수 있습니다.

RED, GREEN 변수를 사용하여 LED의 변화를 확인합니다. LED는 표시는 Alarm을 우선시 합니다.

3.4. 예제 - Servo ID

MightyZAP의 ServoID() 명령어를 이용하여 서보 액추에이터의 ID를 설정합니다.

[예제] - [MightyZAP] - [UNO/LEO]-[MightyZAP_ServoID] 선택

```

MightyZap_ServoID $
#include <MightyZap.h>

MightyZap m_zap(&Serial1, 2);

int ID_Sel =0;

void setup() {
  Serial.begin(9600);
  m_zap.begin(32);
  while (! Serial);
  Serial.print("Input ID : ");
}

void loop() {
  m_zap.ledOn(1,RED);
  m_zap.ledOn(2, GREEN);

  if(Serial.available()) {
    ID_Sel = Serial.parseInt();
    Serial.println(ID_Sel);
    Serial.print("Input_ID [0~2] : ");
    m_zap.ServoID(0, ID_Sel);
    m_zap.ServoID(1, ID_Sel);
    m_zap.ServoID(2, ID_Sel);
  }
}

```

Description

서보 액추에이터의 ID를 변경하여 LED의 변화를 확인합니다.

LedON() 명령어를 이용하여 LED의 색을 제어하고, 서보 액추에이터의 ID를 변경하여 ID의 변화에 따른 LED의 색변화를 확인합니다.

이후 예제는 모두 ID 0번으로 되어있어 ID를 다시 0번으로 변경합니다.

3.5. 예제 - Goal Position

MightyZAP의 goalPosition() 명령어를 이용하여 서보 액추에이터의 위치를 제어하는 예제 입니다.

[예제] - [MightyZAP] - [UNO/LEO]-[MightyZAP_GoalPosition] 선택



```

MightyZap_GoalPosition $
#include <MightyZap.h>
#define ID_NUM 0

MightyZap m_zap(&Serial1, 2);

void setup() {
  m_zap.begin(32);
}

void loop() {
  m_zap.GoalPosition(ID_NUM, 0);
  delay(1000);
  m_zap.GoalPosition(ID_NUM, 4095);
  delay(1000);
}

```

Description

MightyZAP API중 GoalPosition 명령을 테스트 합니다.

1 초 간격으로 0 ~ 4095의 위치 값으로 움직이는 서보 액추에이터를 확인합니다.

m_zap.GoalPosition(ID_NUM, 위치값) 함수에 위치 값을 변경하여 서보 액추에이터의 위치가 변경되는 것을 확인합니다.

3.6. 예제 - Present Position

MightyZAP의 presentPosition() 명령어를 이용하여 서보 액츄에이터의 현재 위치를 확인하는 예제 입니다.

[예제] - [MightyZAP] - [UNO/LEO]-[MightyZAP_ presentPosition] 선택

```

MightyZap_PresentPosition

#include <MightyZap.h>
#define ID_NUM 0

MightyZap m_zap(8Serial1, 2);

int Position;
int cPosition;
int Display =1;

void setup() {
  Serial.begin(9600);
  m_zap.begin(32);
  while (!Serial);
  m_zap.GoalSpeed(ID_NUM,50);
}

void loop() {
  if(Display == 1){
    Serial.print("New Position[0~4095] : ");
    Display = 0;
  }
  if(Serial.available()) {
    Position = Serial.parseInt();
    Serial.println(Position);
    delay(200);

    m_zap.GoalPosition(ID_NUM,Position);
    delay(50);
    while(m_zap.Moving(ID_NUM)) {
      cPosition = m_zap.presentPosition(ID_NUM);
      Serial.print(" - Position : ");
      Serial.println(cPosition);
    }
    delay(200);
    cPosition = m_zap.presentPosition(ID_NUM);
    Serial.print(" - final Position : ");
    Serial.println(cPosition);
    Display = 1;
  }
}

```

Description

MightyZAP API중 PresentPosition명령을 테스트 합니다.

아두이노의 시리얼 모니터를 열고 이동하고자 하는 위치 값을 입력하여 모터의 움직임을 실시간으로 확인합니다.

m_zap.GoalPosition()명령어로 서보 액츄에이터의 위치를 변경 시킨 후 m_zap.presentPosition() 명령어로 서보 액츄에이터의 실시간 위치를 확인합니다.

m_zap.moving() 함수를 이용하여 서보 액츄에이터의 동작상태를 확인합니다.

(1 : 동작, 0: 동작 정지)

3.7. 예제 -GoalSpeed[FC Version Only]

MightyZAP의 GoalSpeed() 명령어를 이용하여 서보 액추에이터의 목표 속도를 설정하는 예제 입니다.

[예제] - [MightyZAP] - [UNO/LEO]-[MightyZAP_GoalSpeed] 선택

MightyZap_GoalSpeed \$

```
#include <MightyZap.h>
#define ID_NUM 0

MightyZap m_zap(8Serial1, 2);
int Speed, Display=1, cPosition;

void setup() {
  Serial1.begin(9600);
  m_zap.begin(32);
  while (!Serial1);
}

void loop() {
  if(Display ==1) {
    m_zap.GoalPosition(ID_NUM,0);
    delay(500);
    while(m_zap.Moving(ID_NUM)){
      cPosition = m_zap.presentPosition(ID_NUM);
      Serial1.print(" - Position : ");
      Serial1.println(cPosition);
    }
    m_zap.GoalPosition(ID_NUM,4095);
    delay(100);
    while(m_zap.Moving(ID_NUM)){
      cPosition = m_zap.presentPosition(ID_NUM);
      Serial1.print(" - Position : ");
      Serial1.println(cPosition);
    }
    Serial1.print("New Speed[0~1023] : ");
    Display=0;
  }
  if(Serial1.available()) {
    Speed = Serial1.parseInt();
    m_zap.GoalSpeed(ID_NUM, Speed);
    Serial1.println(m_zap.GoalSpeed(ID_NUM));
    delay(500);
    Display=1;
  }
}
```

Description

MightyZAP API중GoalSpeed명령을 테스트 합니다.

아두이노의 시리얼 모니터를 실행 시킨 후 서보 액추에이터의 목표 속도를 변경하여 속도 변화를 확인합니다.

또한, 그 변화를 moving() 명령을 이용하여 서보 액추에이터의 동작 상태를 확인하여 모터가 동작되는 동안 presensPosition() 명령을 사용하여 위치 값을 모니터링 한다.

3.8. 예제 -GoalCurrent[FC Version Only]

MightyZAP의 GoalCurrent() 명령어를 이용하여 서보 액추에이터의 최대 전류값을 설정하는 예제 입니다.

[예제] - [MightyZAP] - [UNO/LEO]-[MightyZAP_GoalCurrent] 선택

MightyZap_GoalCurrent\$

```
#include <MightyZap.h>
#define ID_NUM 0

MightyZap m_zap(&Serial1, 2);
int Current, Display=1, cPosition;

void setup() {
  Serial.begin(9600);
  m_zap.begin(32);
  while (!Serial);
}

void loop() {
  if(Display ==1) {
    m_zap.GoalPosition(ID_NUM,0);
    delay(100);
    while(m_zap.Moving(ID_NUM)){
      cPosition = m_zap.presentPosition(ID_NUM);
      Serial.print(" - Position : ");
      Serial.println(cPosition);
    }
    m_zap.GoalPosition(ID_NUM,4095);
    delay(100);
    while(m_zap.Moving(ID_NUM)){
      cPosition = m_zap.presentPosition(ID_NUM);
      Serial.print(" - Position : ");
      Serial.println(cPosition);
    }
    Serial.print("New Current[0~1023] : ");
    Display=0;
  }
  if(Serial.available()) {
    Current = Serial.parseInt();
    m_zap.GoalCurrent(ID_NUM,Current);
    Serial.println(m_zap.GoalCurrent(ID_NUM));
    delay(500);
    Display=1;
  }
}
```

Description

MightyZAP API중 GoalCurrent명령을 테스트 합니다.

서보 액추에이터의 최대 전류값을 변경하고 위치 이동 명령으로 확인합니다.

M_zap.GoalCurrent() 명령으로 최대 전류값을 설정한 후 M_zap.GoalPosition() 명령을 이용하여 그 변화를 확인한다.

GoalCurrent()값에 따라 속도와 모터의 Force가 변경됩니다.

또한, 그 변화를 moving() 명령과 presentPosition() 명령을 사용하여 위치 값을 모니터링 한다.

3.9. 예제 – Stroke Limit

MightyZAP의 LongStrokeLimit()와ShortStrokeLimit() 명령어를 이용하여 서보 액추에이터의 위치 한계값을 설정하는 예제 입니다.[예제] - [MightyZAP] - [UNO/LEO]-[MightyZAP_StrokeLimit] 선택

```

MightyZap_StrokeLimit$
#include <MightyZap.h>
#define ID_NUM 0

MightyZap m_zap(&Serial1, 2);

void setup() {
  m_zap.begin(32);
}

void loop() {
  m_zap.LongStrokeLimit(ID_NUM, 3095);
  m_zap.GoalPosition(ID_NUM, 4095);
  delay(5000);

  m_zap.ShortStrokeLimit(ID_NUM, 100);
  m_zap.GoalPosition(ID_NUM, 0);
  delay(5000);
}

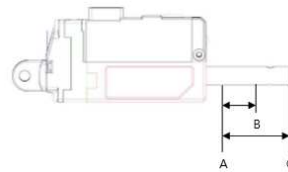
```

Description

MightyZAP API중 ShortStrokeLimit 명령과 LongStrokeLimit 명령을 테스트 합니다.

변경된 Strokelimit 값에 따른 모터의 위치 변화량을 확인합니다.

- ※ Shot Stroke(A) : 수축방향
- Long Stroke(C) : 확장방향



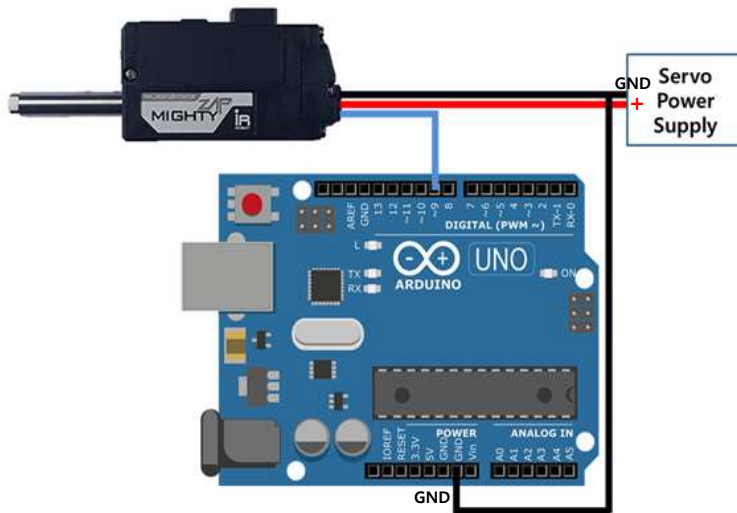
4

아두이노 IDE 를 통한 서보 PWM 예제

아두이노 IDE를 통해 PWM으로 서보 액추에이터를 제어합니다. 서보 액추에이터를 PWM으로 직접 위치 제어할 때 사용합니다.

4.1.개요 / 주의 사항

MightyZAP을 아두이노로 직접 제어하기 위해서는 아래와 같이 회로를 구성해야 합니다. 이때 서보 액추에이터의 전압은 사양에 맞게 입력해야 합니다.



서보 액추에이터에는 전원, 접지 신호의 3개의 선이 있습니다. 전원 선은 빨간색, 접지 선은 검정색, 신호 선은 흰색입니다. 서보 액추에이터의 전원은 해당 재원에 따라 따로 공급되어집니다(7V/12V).

접지 선은 서보 액추에이터 접지와 아두이노의 접지가 연결되어야 합니다.

신호선은 10번에 연결합니다.

4.2. PWM을 이용한 위치제어

아두이노의 PWM 라이브러리로 서보 액추에이터의 위치를 제어합니다.

```
BLE_Name_Setting $
#include <Servo.h>
Servo myservo;

void setup() {
  myservo.attach(10);
}

void loop() {
  myservo.writeMicroseconds(1200);
  delay(4000);

  myservo.writeMicroseconds(1800);
  delay(4000);
}
```

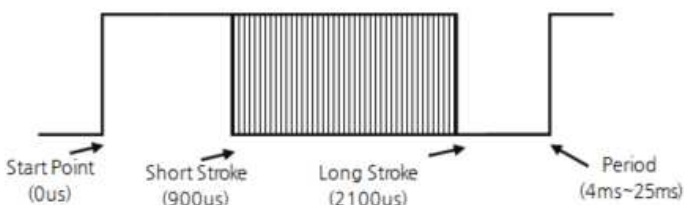
`#include <Servo.h>` :서보라이브러리를 사용하기 위해 헤더 파일을 선언합니다.

`Servo myservo` :서보 액추에이터를 제어하기 위한 이름의 변수를 정의합니다.

`Servo.attach(servoPin)` : servo 변수에 실제로 제어할 서보의제어핀을 지정합니다.

`Servo.writeMicroseconds()` :PWMPulse폭을 us 단위로 입력합니다. (범위 900 ~ 2100us)

➔ `Servo.write()` :일반 RC 서보의 제어방법으로 0 ~ 180의 값을 넣어 길이를 제어합니다.



Rod Stroke	Pulse Mode
Short Stroke	900us
Half Stroke	1500us
Long Stroke (27mm사용시)	2100us
Long Stroke (30mm사용시)	2100us

[참고 - 아두이노테스트 쉴드 IR-STS01]

아이알로봇에서 제공하는 아두이노서보 테스트 쉴드를 사용하면 보다 편리하게 아두이노 제어가 가능합니다. 아두이노레오나르도와 당사 자체 제작 서보쉴드로 구성이 되어 있으며, 아두이노서보 테스트 쉴드를 통해 복잡한 배선 작업 없이 곧바로 마이티잡서보를 연결할 수 있고, 즉시 제어도 가능합니다. 또한 여러 스위치를 통해 마이티잡의 운동 리미트, 속도, 신호 delay 등을 설정할 수 있으며, 쉴드 전용 아두이노 API 도 공개하고 있습니다.

구체적인 제품 설명은 별도의 아두이노테스트쉴드(IR-STS01) 매뉴얼을 참고하여 주십시오.

